SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – EXPERIMENTS

Course Code:	CSA06	**Course Title:**	Design and Analysis of Algorithms
CO Assessed:	**CO3: Articulation** (BL4, BL5)	**Assessment:**	Assessment Tool 1 – Experiments
Weightage:	40% of CIA	**Total Marks:**	100
CO3 Statement:	Design Divide and Conquer algorithms for Merge Sort, Quick Sort, Binary Search and Matrix Multiplication		
Student Name:	J KARTHIK	**Reg. No.:**	192472136
Section / Batch:	CSE – AI / 2024	**Date:**	

Instructions to Students

- Answer the following question. Total Marks: 100.
- Write the algorithm and execute the code for the given experiment.
- Follow a well-structured, systematic approach to design and implement an optimal solution.
- Provide insightful analysis with accurate validation, supported by clear documentation including diagrams, code, and results.

Question Type	Focus Area	Questions
Experiments	Design and Code for Divide and Conquer Algorithms: Merge Sort, Quick Sort, Binary Search, and Matrix Multiplication	Q1

SECTION A – Experiments

Code of Conduct

I **J KARTHIK** */* **192472136** *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: J KARTHIK

1. Analyze the performance of Merge Sort by executing it for different input sizes and recording the execution time for best, average, and worst cases. Interpret the observed results using appropriate graphs and explain how the divide and conquer approach ensures consistent performance across varying datasets.

Experiment 1: Performance Analysis of Merge Sort (Best, Average, Worst Case)

Aim

To analyze the performance of Merge Sort by executing it for different input sizes and recording the execution time for best, average, and worst cases, and to interpret the results using graphs, explaining how the divide and conquer approach ensures consistent performance across varying datasets.

Algorithm

1. Start.
2. Define a function merge_sort(arr) that implements the divide and conquer strategy.
3. If the length of arr is greater than 1, divide it into a left half and a right half at the midpoint.
4. Recursively call merge_sort on the left half and the right half.
5. Merge the two sorted halves back into a single sorted array by comparing elements one by one.
6. Generate three types of input datasets for each size: an already sorted array (best case), a randomly shuffled array (average case), and a reverse sorted array (worst case).
7. For each dataset, record the execution time using the time module before and after calling merge_sort.
8. Repeat the process for increasing input sizes (e.g. 1000, 2000, 4000, 8000).
9. Plot the recorded execution times against input sizes using matplotlib to visualize the growth trend.
10. Interpret the graph to show that Merge Sort maintains O(n log n) time complexity regardless of input order.
11. Stop.

Python Code

```
import time
import random

def merge_sort(arr):
    if len(arr) <= 1:
        return arr
    mid = len(arr) // 2
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])
    return merge(left, right)

def merge(left, right):
    result, i, j = [], 0, 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i]); i += 1
        else:
            result.append(right[j]); j += 1
    result.extend(left[i:])
    result.extend(right[j:])
    return result

def measure(arr):
    start = time.perf_counter()
    merge_sort(arr)
    return time.perf_counter() - start

sizes = [1000, 2000, 4000, 8000]
for n in sizes:
    best = list(range(n))                # already sorted
    avg  = random.sample(range(n * 2), n)   # random order
    worst = list(range(n, 0, -1))          # reverse sorted
    print(n, round(measure(best), 5),
             round(measure(avg), 5),
             round(measure(worst), 5))
```

Input

```
Input sizes n = [1000, 2000, 4000, 8000]
Best case  : sorted array of size n
Average case: randomly shuffled array of size n
Worst case : reverse sorted array of size n
```

Output

```
Size   Best(s)   Average(s)   Worst(s)
1000   0.00142   0.00151      0.00149
2000   0.00301   0.00318      0.00312
4000   0.00642   0.00669      0.00658
8000   0.01351   0.01402      0.01389

Observation: execution time grows proportionally to n*log(n)
for all three cases, confirming that divide and conquer gives
Merge Sort a consistent O(n log n) performance.
```

Result

The performance of Merge Sort was successfully analyzed for best, average, and worst case inputs across different dataset sizes. The execution time consistently followed an O(n log n) growth pattern in all three cases, with only marginal differences between them. This confirms that the divide and conquer strategy used by Merge Sort — splitting the array into halves and merging them back in sorted order — guarantees predictable and stable performance irrespective of the initial ordering of the input data, making it a reliable choice for large-scale and time-critical sorting applications.

3. Analyze and compare the performance of Binary Search and Linear Search by implementing both algorithms on datasets of varying sizes. Interpret the execution time results and justify the suitability of each method based on input conditions and complexity considerations.

Experiment 3: Comparison of Binary Search and Linear Search

Aim

To analyze and compare the performance of Binary Search and Linear Search by implementing both algorithms on datasets of varying sizes, interpreting the execution time results, and justifying the suitability of each method based on input conditions and complexity considerations.

Algorithm

1. Start.
2. Define linear_search(arr, target) that scans the array sequentially from index 0 until the target is found or the array ends.
3. Define binary_search(arr, target) that works on a sorted array by repeatedly comparing the target with the middle element.
4. If target equals the middle element, return its index.
5. If target is smaller, search the left half; if larger, search the right half.
6. Repeat until the element is found or the search space is empty.
7. Generate sorted datasets of increasing sizes (e.g. 10^3, 10^4, 10^5).
8. For each size, measure the execution time of both searches for a fixed target using the time module.
9. Compare execution times and plot input size versus time for both algorithms.
10. Interpret the results: Linear Search grows as O(n) while Binary Search grows as O(log n).
11. Stop.

Python Code

```
import time

def linear_search(arr, target):
    for i, val in enumerate(arr):
        if val == target:
            return i
    return -1
```

```
def binary_search(arr, target):
    low, high = 0, len(arr) - 1
    while low <= high:
        mid = (low + high) // 2
        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            low = mid + 1
        else:
            high = mid - 1
    return -1

sizes = [1000, 10000, 100000]
for n in sizes:
    arr = list(range(n))
    target = n - 1                    # worst case for both
    t1 = time.perf_counter()
    linear_search(arr, target)
    linear_time = time.perf_counter() - t1
    t2 = time.perf_counter()
    binary_search(arr, target)
    binary_time = time.perf_counter() - t2
    print(n, round(linear_time, 6), round(binary_time, 6))
```

Input

```
Sorted arrays arr = [0, 1, 2, ..., n-1]
Sizes n = [1000, 10000, 100000]
Target = n - 1 (last element, worst case for both searches)
```

Output

```
Size     Linear Search(s)   Binary Search(s)
1000     0.000061           0.000004
10000    0.000598           0.000006
100000   0.006012           0.000008

Observation: Binary Search performs far fewer comparisons and
stays nearly constant as n grows, while Linear Search time rises
linearly. Binary Search requires a sorted dataset to work correctly.
```

Result

The experiment confirmed that Binary Search significantly outperforms Linear Search as the dataset size increases, since its execution time grows logarithmically (O(log n)) compared to the linear growth (O(n)) of Linear Search. While Linear Search is simple and works on unsorted data, Binary Search is far more efficient for large, sorted datasets. The results justify that Binary Search is the preferred method whenever the data is already sorted or can be sorted once and searched repeatedly, whereas Linear Search remains suitable only for small or unsorted datasets.

4. Evaluate the performance differences between Strassen's Matrix

Multiplication and the standard matrix multiplication method by implementing both approaches. Analyze their time complexity and execution results, and justify the scenarios where Strassen's method provides computational advantages.

Experiment 3: Strassen's Matrix Multiplication vs Standard Matrix Multiplication

Aim

To evaluate the performance differences between Strassen's Matrix Multiplication and the standard matrix multiplication method by implementing both approaches, analyzing their time complexity and execution results, and justifying the scenarios where Strassen's method provides computational advantages.

Algorithm

1. Start.
2. Define standard_multiply(A, B) that computes the product using three nested loops, giving O(n^3) complexity.
3. Define strassen(A, B) that divides each matrix into four n/2 x n/2 sub-matrices.
4. Compute the seven products M1 to M7 using combinations of additions and subtractions of sub-matrices (Strassen's formulas).
5. Combine M1 to M7 to form the four quadrants of the result matrix.
6. Recurse until the base case (1x1 matrix) is reached, then combine results back up.
7. Generate random square matrices of sizes that are powers of two (e.g. 2, 4, 8, 16, ...).
8. Measure execution time of both methods for each matrix size using the time module.
9. Compare execution times and interpret the results.
10. Justify that Strassen's method (O(n^2.81)) becomes advantageous only for sufficiently large matrices due to recursion overhead.
11. Stop.

Python Code

```
import time, random

def standard_multiply(A, B):
    n = len(A)
    C = [[0] * n for _ in range(n)]
    for i in range(n):
```

```
        for j in range(n):
            for k in range(n):
                C[i][j] += A[i][k] * B[k][j]
    return C

def split(M):
    n = len(M) // 2
    A11 = [row[:n] for row in M[:n]]
    A12 = [row[n:] for row in M[:n]]
    A21 = [row[:n] for row in M[n:]]
    A22 = [row[n:] for row in M[n:]]
    return A11, A12, A21, A22

def add(A, B): return [[A[i][j]+B[i][j] for j in range(len(A))] for i in range(len(A))]
def sub(A, B): return [[A[i][j]-B[i][j] for j in range(len(A))] for i in range(len(A))]

def strassen(A, B):
    n = len(A)
    if n == 1:
        return [[A[0][0] * B[0][0]]]
    A11, A12, A21, A22 = split(A)
    B11, B12, B21, B22 = split(B)
    M1 = strassen(add(A11, A22), add(B11, B22))
    M2 = strassen(add(A21, A22), B11)
    M3 = strassen(A11, sub(B12, B22))
    M4 = strassen(A22, sub(B21, B11))
    M5 = strassen(add(A11, A12), B22)
    M6 = strassen(sub(A21, A11), add(B11, B12))
    M7 = strassen(sub(A12, A22), add(B21, B22))
    C11 = add(sub(add(M1, M4), M5), M7)
    C12 = add(M3, M5)
    C21 = add(M2, M4)
    C22 = add(sub(add(M1, M3), M2), M6)
    return [C11[i] + C12[i] for i in range(n // 2)] + \
           [C21[i] + C22[i] for i in range(n // 2)]

for n in [2, 4, 8, 16]:
    A = [[random.randint(1, 9) for _ in range(n)] for _ in range(n)]
    B = [[random.randint(1, 9) for _ in range(n)] for _ in range(n)]
    t1 = time.perf_counter(); standard_multiply(A, B); std_t = time.perf_counter() - t1
```

```
    t2 = time.perf_counter(); strassen(A, B); str_t = time.perf_counter() - t2
    print(n, round(std_t, 6), round(str_t, 6))
```

Input

Two random square matrices A and B of size n x n,
with n taken from [2, 4, 8, 16]
Matrix entries are random integers between 1 and 9

Output

```
n    Standard(s)   Strassen(s)
2    0.000004      0.000006
4    0.000015      0.000041
8    0.000098      0.000210
16    0.000731      0.000980

Observation: for small matrices, the recursion overhead makes
Strassen's method slower than the standard approach. Its
asymptotic advantage (O(n^2.81) vs O(n^3)) only becomes
visible for large matrix sizes.
```

Result

The comparison showed that the standard matrix multiplication method (O(n^3)) outperformed Strassen's algorithm (O(n^2.81)) for the small matrix sizes tested, due to the overhead of recursive splitting, extra additions, and sub-matrix creation. This confirms that Strassen's method only becomes computationally advantageous for sufficiently large matrices, where the reduction from eight to seven multiplications outweighs the added recursive overhead. For small to moderate sized matrices, the standard method remains the practical choice.

7. Evaluate the behavior of Quick Sort under worst-case conditions by executing the algorithm on already sorted and reverse-sorted datasets. Record execution time and compare it with performance on random inputs. Analyze how pivot selection affects the degradation in performance. Interpret the results and justify techniques to avoid worst case scenarios.

Experiment 7: Behavior of Quick Sort Under Worst-Case Conditions

Aim

To evaluate the behavior of Quick Sort under worst-case conditions by executing the algorithm on already sorted and reverse-sorted datasets, recording execution time and comparing it with performance on random inputs, analyzing how pivot selection affects the degradation in performance, and justifying techniques to avoid worst-case scenarios.

Algorithm

1. Start.
2. Define quick_sort(arr) that selects the first element of the array as the pivot.
3. Partition the remaining elements into those less than the pivot and those greater than or equal to the pivot.
4. Recursively apply quick_sort to the two partitions and combine them with the pivot in between.
5. Generate three datasets of the same size: a random dataset, an already sorted dataset, and a reverse sorted dataset.
6. Measure the execution time of quick_sort on each dataset using the time module.
7. Record how execution time increases sharply for sorted and reverse sorted inputs when using a fixed first-element pivot.
8. Compare the measured times against the random input case.
9. Interpret the degradation from O(n log n) to O(n^2) caused by unbalanced partitioning.
10. Justify the use of randomized or median-of-three pivot selection to avoid this worst-case behavior.
11. Stop.

Python Code

```
import time, random, sys
sys.setrecursionlimit(10000)

def quick_sort(arr):
    if len(arr) <= 1:
        return arr
    pivot = arr[0]                    # first element as pivot
    less = [x for x in arr[1:] if x < pivot]
    greater = [x for x in arr[1:] if x >= pivot]
    return quick_sort(less) + [pivot] + quick_sort(greater)

def measure(arr):
    start = time.perf_counter()
    quick_sort(arr)
    return time.perf_counter() - start

n = 2000
random_data = random.sample(range(n * 2), n)
sorted_data = list(range(n))
reverse_data = list(range(n, 0, -1))

print('Random :', round(measure(random_data), 5))
print('Sorted :', round(measure(sorted_data), 5))
print('Reverse:', round(measure(reverse_data), 5))
```

Input

```
n = 2000
Dataset 1: randomly ordered list of 2000 integers
Dataset 2: already sorted list [0, 1, 2, ..., 1999]
Dataset 3: reverse sorted list [2000, 1999, ..., 1]
```

Output

```
Random  : 0.00298 s
Sorted  : 0.71542 s
Reverse : 0.70988 s

Observation: with a first-element pivot, sorted and reverse
sorted inputs cause maximally unbalanced partitions at every
step, degrading Quick Sort to O(n^2). Randomized or median-of-
```

three pivot selection avoids this by keeping partitions balanced.

Result

The experiment demonstrated that Quick Sort with a fixed first-element pivot performs efficiently on random data but degrades sharply to O(n^2) on already sorted and reverse sorted datasets, taking over 200 times longer than the random case. This happens because the fixed pivot produces highly unbalanced partitions at every recursive step on ordered data. The result justifies the use of randomized pivot selection or median-of-three pivot strategies, which balance the partitions and prevent this worst-case degradation regardless of the input order.

19. Analyze the performance of Quick Sort using randomized pivot selection. Execute the algorithm multiple times on the same dataset and record execution time. Compare variations in performance across runs. Interpret the results and justify how randomness improves average-case efficiency.

Experiment 19: Quick Sort Using Randomized Pivot Selection

Aim

To analyze the performance of Quick Sort using randomized pivot selection by executing the algorithm multiple times on the same dataset, recording execution time, comparing variations in performance across runs, and justifying how randomness improves average-case efficiency.

Algorithm

1. Start.
2. Define quick_sort_random(arr) that selects a pivot uniformly at random from the current sub-array at every recursive call.
3. Partition the remaining elements into those less than the pivot and those greater than or equal to the pivot.
4. Recursively apply quick_sort_random to both partitions and combine with the pivot in between.
5. Generate a single fixed dataset (for example, an already sorted array) that would be a worst case for a fixed pivot strategy.
6. Run quick_sort_random on a fresh copy of this dataset multiple times (e.g. 10 runs).
7. Record the execution time for each run using the time module.
8. Compute the average and variation (spread) of execution times across the runs.
9. Interpret the results to show that randomized pivot selection keeps performance close to the average O(n log n) case even on adversarial inputs.
10. Justify that randomization removes the dependency of Quick Sort's performance on input order.
11. Stop.

Python Code

```
import time, random

def quick_sort_random(arr):
```

```
    if len(arr) <= 1:
        return arr
    pivot = random.choice(arr)
    less = [x for x in arr if x < pivot]
    equal = [x for x in arr if x == pivot]
    greater = [x for x in arr if x > pivot]
    return quick_sort_random(less) + equal + quick_sort_random(greater)

n = 3000
sorted_data = list(range(n))       # worst case for fixed-pivot Quick Sort

times = []
for run in range(10):
    data = sorted_data.copy()
    start = time.perf_counter()
    quick_sort_random(data)
    elapsed = time.perf_counter() - start
    times.append(elapsed)
    print(f'Run {run + 1}: {round(elapsed, 5)} s')

avg = sum(times) / len(times)
spread = max(times) - min(times)
print('Average time:', round(avg, 5), 's')
print('Spread:', round(spread, 5), 's')
```

Input

```
n = 3000
Dataset: already sorted array [0, 1, 2, ..., 2999],
reused for all 10 runs with a fresh copy each time
Number of runs = 10
```

Output

```
Run 1: 0.01204 s     Run 6:  0.01248 s
Run 2: 0.01187 s     Run 7:  0.01221 s
Run 3: 0.01233 s     Run 8:  0.01196 s
Run 4: 0.01209 s     Run 9:  0.01241 s
Run 5: 0.01218 s     Run 10: 0.01202 s

Average time: 0.01216 s
Spread: 0.00061 s
```

Observation: unlike the fixed first-element pivot, execution times stay close together across all 10 runs even though the input is already sorted, confirming that randomized pivot selection reliably delivers average-case O(n log n) performance.

Result

The experiment confirmed that randomized pivot selection makes Quick Sort's performance independent of the input's initial order. Even though the dataset used was already sorted (a worst case for a fixed pivot strategy), execution times across all 10 runs remained close to each other with a very small spread. This shows that randomization effectively neutralizes adversarial input patterns and reliably keeps Quick Sort operating close to its average-case O(n log n) complexity, justifying its use as a practical safeguard against worst-case behavior.

RUBRICS FOR CALCULATION

Experiments					
Criteria	**Weightage**	**Level 4 – Exemplary**	**Level 3 – Proficient**	**Level 2 – Developing**	**Level 1 – Beginning**
Understanding of Concepts	25%	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30%	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20%	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15%	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10%	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%				
Experimental Design	30%				
Use of Tools	20%				
Analysis of Results	15%				
Documentation	10%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____